Symbol:

- A symbol (often also called a character) is the smallest building block, which can be any alphabet, letter, or picture. **symbol** is an indivisible unit or entity used to construct strings.

- It is a member of a **finite alphabet** (denoted as Σ), which is the set of allowable symbols that a computational model can work with. a,b,c,…..z,1,2,...9, +, -, …

Examples of Symbols:

- In the binary alphabet Σ = {0, 1}, the symbols are "0" and "1".
- In the alphabet Σ = {a, b, c}, the symbols are "a", "b", and "c".
- In a natural language alphabet, such as the English alphabet, the symbols are the 26 letters {A, B, C, ..., Z}.

Types of Symbols in TOC:

- **Input Symbols**: The symbols that a machine or automaton reads as input from the alphabet.
- **Tape Symbols** (for Turing machines): These include the input symbols, as well as additional symbols that can be written on the tape during computation.

a, b, c, 0, 1, ...

Alphabet:

- Finite non-empty set of symbol. Denoted by sigma (**Σ**). Alpabets an **alphabet** is a fundamental concept. It refers to a finite set of symbols, typically represented by letters, that are used to construct strings, which in turn are processed by formal languages and automata.

Alphabet (Σ):

- Denoted by Σ (sigma), an alphabet is a non-empty, finite set of symbols. Each symbol in an alphabet is atomic, meaning it cannot be broken down further within the context of the language.
- For example, Σ = {0, 1} represents a binary alphabet, and Σ = {a, b, c} represents an alphabet with three characters.

- Σ = {0,1} set of binary alphabets.
- Σ = {a,b,c,……..,z} set of all lower case letters.
- Σ = {A,B,C,……..Z} set of all upper case letters.
- Σ = {+,&,%,…….} set of all special characters.
- **Σ** = {a,b,c...z} set of all lower case letters.

Examples:

$\Sigma = \{0, 1\}$ is an alphabet of binary digits

$\Sigma = \{0, 1, \ldots\ldots , 9\}$ is an alphabet of decimal digits

$\Sigma = \{a, b, c\}$

$\Sigma = \{A, B, C, \ldots\ldots\ldots, Z\}$

String:

- A string is **a finite set sequence of symbols** choosen from some alphabets. A string is generally denoted as *w* and the length of a string is denoted as $|w|$.
- 0110111 taken from {0,1}.
- abaababab taken from {a,b}.

- **Empty String**: The empty string is the string with no symbols. It is denoted by ε.

- **Length of string:** It is the number of symbols in the string. It is denoted by |W|.

- **Concatenation of string:** Join 2 or more strings.

Power of alphabet:

- If **Σ** is an aplhabet, we can express set of all strings of certain length from that alphabet by using exponential notation. It is denoted by ε^k , where k is the length of the string.

- **Σ** = {0,1}….only 2 symbols => {0,1}

- Σ^2 => {00, 01,10,11}

- Σ^3 => {000, 001, 010, 011, 100, 101, 110, 111}

Kleen Closure:

- The set of strings (including ephsilon, ε) over an aplhabet is usually dentoed by Σ^*. Kleene Star is also called a *"Kleene Operator"* or *"Kleene Closure"*.

- Let Σ^* = $\{0,1\}^*$ => {ε, 0,1,00,11, 10, 01,……}
  ⇨ Σ^* = E0 U E1 U E2 U E3……

Kleen Plus OR Positive Closure:

- The set of strings (excluding ephsilon) over an aplhabet is usually dentoed by Σ^+.

- Σ^+ = {0,1,01,10,00,11……}

- $\Sigma^+ = \Sigma^* - \varepsilon$

Power of alphabets:

- It is of 2 types:-
  1. Kleen Closure
  2. Kleen Plus

Language:

- A language is a *set of strings*, chosen from some Σ* or we can say- "A language is a subset of Σ* ". A language that can be formed over ' Σ ' can be Finite or Infinite.

- If **Σ** is an alphabet, and L belongs to **Σ*** then L is a language.

```
Example of Finite Language:
        L1 = { set of string of 2 }
    L1 = { xy, yx, xx, yy }

Example of Infinite Language:
    L1 = { set of all strings starts with 'b' }
    L1 = { babb, baa, ba, bbb, baab, ..... }
```

Operations on language:

1. **Complimentation of a language:** Let L be any language over alphabet E, The complimentation of L, deonted by L', L'= **Σ***- L.
2. **Union of L1 and L2 =** L1 U L2.
3. **Intersection of L1 and L2 =** L1 n L2.
4. **Concatenation =** L1.L2, w1.w2…. Where w1 is in L1 and w2 is in L2. w1.w2 ≠ w2.w1.
5. **Reversal =** L^r = {W^r where W ∈ L}.

Derivation:

- Derivation is a sequence of production rules. It is used to get the input string through these production rules. During parsing, we have to take two decisions. These are as follows:
- We have to decide the non-terminal which is to be replaced.

- We have to decide the production rule by which the non-terminal will be replaced.
- We have two options to decide which non-terminal to be placed with production rule.

---

1. Leftmost Derivation:

- In the leftmost derivation, the input is scanned and replaced with the production rule from left to right. So in leftmost derivation, we read the input string from left to right.

Example:

- **Production rules:**

  E = E+ E

  E= E – E
  E=a | b

- **Input:** a – b + a

- **The leftmost derivation is:**

  E= E + E

  E = E – E+ E
  E= a – E + E
  E = a – b + E
  E = a – b + a

2. Rightmost Derivation:

In rightmost derivation, the input is scanned and replaced with the production rule from right to left. So in rightmost derivation, we read the input string from right to left.

Example:

- **Production rules:**

  E = E + E

  E = E – E
  E = a | b

- Input : a – b+ a

- The rightmost derivation is:
  E = E – E
  E = E – E + E
  E= E – E + a
  E = E – b + a
  E = a – b + a

- **When we use the leftmost derivation or rightmost derivation, we may get the same string. This type of derivation does not affect on getting of a string.**

Examples of Derivation:

Example 1:

Derive the string "abb" for leftmost derivation and rightmost derivation using a CFG given by,

S→AB | ε
A→aB
B→Sb

Solution:

Leftmost derivation:

S

AB

aB B

a Sb B

A ε bB

ab Sb

ab ε b

abb

Rightmost derivation:

S

AB

A Sb

A ε b

aB b

a Sb b

a ε bb

abb

Example 2:

Derive the string "aabbabba" for leftmost derivation and rightmost derivation using a CFG given by,

S→aB | bA
S→a|aS|bAA
S→b|aS|aBB

Solution:

Leftmost derivation:

1. S
2. aBS→aB
3. aaBBB→aBB
4. aabB B → b
5. aabbS B → bS
6. aabbaB S → aB
7. aabbabS B → bS
8. aabbabbA S → bA
9. aabbabba A → a

Rightmost derivation:

1. S
2. aB S → aB
3. aaBB B → aBB
4. aaBbS B → bS
5. aaBbbA S → bA
6. aaBbba A → a
7. aabSbba B → bS
8. aabbAbba S → bA
9. aabbabba A → a

Example 3:

Derive the string "00101" for leftmost derivation and rightmost derivation using a CFG given by,

$S \rightarrow A1B$
$A \rightarrow 0A \mid \varepsilon$
$B \rightarrow 0B \mid 1B \mid \varepsilon$

Solution:

Leftmost derivation:

1. S
2. A1B
3. 0A1B
4. 00A1B
5. 001B
6. 0010B
7. 00101B
8. 00101

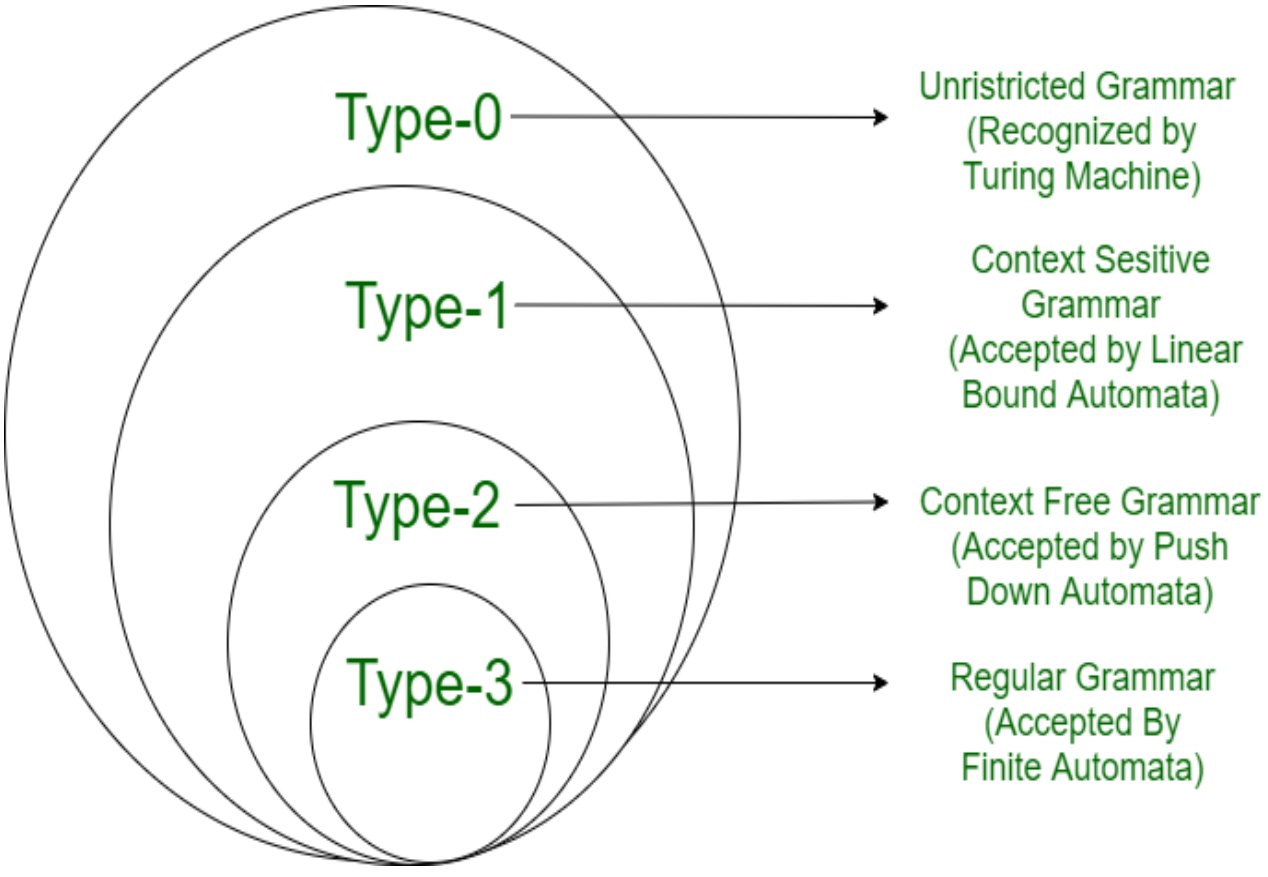
Rightmost derivation:

1. S
2. A1B
3. A10B
4. A101B
5. A101
6. 0A101
7. 00A101
8. 00101

Chomsky Hierarchy:

According to **Chomsky hierarchy**, grammar is divided into 4 types as follows:

- Type 0 is known as unrestricted grammar.
- Type 1 is known as context-sensitive grammar.
- Type 2 is known as a context-free grammar.
- Type 3 Regular Grammar.



Grammar Type	Grammar Accepted	Language Accepted	Automaton
Type 0	Unrestricted grammar	Recursively enumerable language	Turing Machine
Type 1	Context-sensitive grammar	Context-sensitive language	Linear-bounded automaton
Type 2	Context-free grammar	Context-free language	Pushdown automaton
Type 3	Regular grammar	Regular language	Finite state automaton

Type - 3 Grammar:

- **Type-3 grammars** generate regular languages.

- Type-3 grammars must have a single non-terminal on the left-hand side and a right-hand side consisting of a single terminal or single terminal followed by a single non-terminal.

- Type-3 grammars generate regular languages. These languages are exactly all languages that can be accepted by a finite-state automaton.

- Type 3 is the most restricted form of grammar.
- The productions must be in the form **X → a or X → aY**
- where **X, Y ∈ N** (Non terminal) and **a ∈ T** (Terminal).
- The rule **S → ε** is allowed if **S** does not appear on the right side of any rule.

- **Example**
  $X \rightarrow \varepsilon$
  $X \rightarrow a \mid aY$
  $Y \rightarrow b$

Type - 2 Grammar

- **Type-2 grammars** generate context-free languages.
- The productions must be in the form $\mathbf{A \rightarrow \gamma}$
- where $\mathbf{A \in N}$ (Non terminal)
- and $\mathbf{\gamma \in (T \cup N)^*}$ (String of terminals and non-terminals).
- These languages generated by these grammars are be recognized by a non-deterministic pushdown automaton.

- **Example**
  $S \rightarrow X\ a$
  $X \rightarrow a$
  $X \rightarrow aX$
  $X \rightarrow abc$
  $X \rightarrow \varepsilon$

Type - 1 Grammar:

- **Type-1 grammars** generate context-sensitive languages.
- The context sensitive grammar follows the following rules:
  1. The context sensitive grammar may have more than one symbol on the left hand side of their production rules.
  2. The number of symbols on the left-hand side must not exceed the number of symbols on the right-hand side.
  3. The rule of the form $A \rightarrow \varepsilon$ is not allowed unless A is a start symbol. It does not occur on the right-hand side of any rule.

4. The Type 1 grammar should be Type 0. In type 1, production is in the form V → T, where the count of symbol in V is less than or equal to T.

- The productions must be in the form **α A β → α γ β**, where **A ∈ N** (Non-terminal) and **α, β, γ ∈ (T ∪ N)*** (Strings of terminals and non-terminals).

- The strings **α** and **β** may be empty, but **γ** must be non-empty.

- The rule **S → ε** is allowed if S does not appear on the right side of any rule.

- The languages generated by these grammars are recognized by a linear bounded automaton.

Example

AB → AbBc
A → bcA
B → b

Type - 0 Grammar:

- **Type-0 grammars** generate recursively enumerable languages.
- The productions have no restrictions.
- They are any phase structure grammar including all formal grammars.
- They generate the languages that are recognized by a Turing machine.
- The productions can be in the form of **α → β** where **α** is a string of terminals and nonterminals with at least one non-terminal and **α** cannot be null.
- **β** is a string of terminals and non-terminals.

Example:

S → ACaB
Bc → acB
CB → DB
aD → Db